Task 1 :Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Test credentials for your screenshot: username `admin`, password `zomato123`. Try a wrong combo first to show "Access Denied," then the correct one for "Access Granted"

//File Separately attached

Task 2: Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Hint: Use a dictionary to map roles to allowed resources..

a **role-based access control (RBAC)** function-

- Different types of users (**roles**) - customer, restaurant owner, admin, etc. — should only be allowed to touch certain parts of the system (**resources**) — like editing a menu, viewing order history, or opening the admin dashboard.
- `check_access(user_role, resource)` takes in *who* is asking (`user_role`) and *what* they're trying to access (`resource`), and returns `True` if that role is allowed to touch that resource, or `False` if not.

Python code:

```
def check_access(user_role, resource):
  # Map each role to the list of resources it is allowed to access
  role_permissions = {
     'customer': ['menu_view', 'order_history', 'place_order'],
     'restaurant_owner': ['menu_view', 'menu_edit', 'order_history', 'sales_report'],
     'admin': ['menu_view', 'menu_edit', 'order_history', 'sales_report', 'admin_dashboard', 'user_management'],
  }

  allowed_resources = role_permissions.get(user_role, [])
  return resource in allowed_resources


# --- Test cases ---
test_cases = [
  ('customer', 'menu_edit'),
  ('customer', 'order_history'),
  ('restaurant_owner', 'menu_edit'),
  ('restaurant_owner', 'admin_dashboard'),
  ('admin', 'admin_dashboard'),
```

```
    ('delivery_partner', 'order_history'),  # role not in dict at all
]

for role, resource in test_cases:
    result = check_access(role, resource)
    print(f"check_access('{role}', '{resource}') -> {result}")
```

```
C:\Users\srikantm05\Downloads>python check_access.py
check_access('customer', 'menu_edit') -> False
check_access('customer', 'order_history') -> True
check_access('restaurant_owner', 'menu_edit') -> True
check_access('restaurant_owner', 'admin_dashboard') -> False
check_access('admin', 'admin_dashboard') -> True
check_access('delivery_partner', 'order_history') -> False
```

Task 3: Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

```
import random

def two_factor_login(correct_password):
    entered_password = input("Enter password: ")

    if entered_password != correct_password:
        print("Login Failed: Incorrect password")
        return

    # Generate a random 6-digit OTP
    otp = random.randint(100000, 999999)
    print(f"Your OTP is: {otp}")  # In a real system this would be sent via SMS/app, not printed

    entered_otp = input("Enter OTP: ")

    if entered_otp == str(otp):
        print("Login Successful")
    else:
        print("Login Failed: Incorrect OTP")

# Example usage
two_factor_login("mypassword123")
```

```
Enter password: mypassword123
Your OTP is: 320936
Enter OTP: 320936
```

```python
import random

def two_factor_login(correct_password):
    entered_password = input("Enter password: ")

    if entered_password != correct_password:
        print("Login Failed: Incorrect password")
        return

    # Generate a random 6-digit OTP
    otp = random.randint(100000, 999999)
    print(f"Your OTP is: {otp}")  # In a real system this would be sent via SMS/app, not printed

    entered_otp = input("Enter OTP: ")

    if entered_otp == str(otp):
        print("Login Successful")
    else:
        print("Login Failed: Incorrect OTP")

# Example usage
two_factor_login("mypassword123")
```

```
Enter password: mypassword123
Your OTP is: 320936
Enter OTP: 320936
Login Successful
```

Task 4: Given an array of user objects with properties username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

//File Seperately attached

Active Premium Users Checker

Reference date: **2024-06-10**. Showing premium status and last-login activity for a random set of users.

Search by username...

USERNAME	PREMIUM	LAST LOGIN	ACTIVE (7 DAYS)
alice	No	2024-06-10	Active
bob	No	2024-05-24	Inactive
carol	Yes	2024-05-27	Inactive
dave	No	2024-06-08	Active
erin	Yes	2024-05-26	Inactive
frank	No	2024-05-23	Inactive
grace	Yes	2024-05-30	Inactive
heidi	No	2024-05-27	Inactive
ivan	Yes	2024-06-06	Active
judy	No	2024-05-31	Inactive
mallory	No	2024-06-03	Active
niaj	Yes	2024-06-10	Active
olivia	No	2024-05-28	Inactive
peggy	Yes	2024-05-25	Inactive
sybil	No	2024-06-08	Active

"Active" = logged in on or after 2024-06-03 (7 days before the reference date).

Task 5: Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Common access control mistakes (Instagram/Flipkart-style apps)

1. **Hiding the button isn't the same as blocking the action** - if the app only hides the "Delete" button from regular users but doesn't actually check on the server whether they're allowed to delete, someone can still trigger it by other means (like directly calling the app's backend). Fix: always double-check permissions on the server, not just in what the screen shows.
2. **Guessable IDs let people peek at others' stuff** - if your order or profile is just orders/1234, someone could type orders/1235 and see a stranger's order. Fix: the app should check "does this order actually belong to this person?" every time, not just "is this person logged in?"
3. **New accounts getting too much access by default -** sometimes a bug or bad setup accidentally gives new users more permissions than they should have. Fix: start everyone with the least access possible, and only add permissions on purpose.
4. **Old permissions sticking around after a demotion** - if someone's admin rights get removed, but their login session doesn't get refreshed, they might still be able to act like an admin for a while. Fix: force a refresh/logout whenever someone's role changes.
5. **No limit on login attempts** - if there's no cap on how many times someone can try a password or OTP, an attacker can just keep guessing until they get in. Fix: lock the account or slow down repeated failed attempts.